

FORMATION PYTHON

J2

Fonctions et structures de données

Signatures complètes, portée, tris avec key=, dictionnaires, ensembles et le module collections — sur le mini-annuaire.

03-fonctions

04-structures-de-donnees

Objectifs de la journée



Des signatures maîtrisées

Positionnels, nommés, valeurs par défaut, `*args/**kwargs`, annotations.



Portée et contrats

LEGB, le piège du défaut mutable, `assert` sur les bords du barème.



Les 4 structures + collections

liste, dict, set, tuple — et Counter/defaultdict qui les complètent.



Le mini-annuaire

Index par NIF, comptages, dédoublonnage, orphelins, tris multi-clés.

LA SEMAINE

J1

Fondamentaux du langage

J2

Fonctions & structures

J3

Fichiers & robustesse

J4

Objets & pandas

J5

Intégration & projets

Fil rouge : pénalités + annuaire.

La signature : le contrat d'une fonction



Positionnels puis nommés

penalite(8_100_000, jours=45) — l'argument nommé documente l'appel.



Valeurs par défaut = options

plafond: float = 0.50 : l'appelant ne précise que l'exceptionnel.



Annotations : doc vivante

def f(ca: int) -> str — VS Code complète, le relecteur comprend, sans coût.



Sans return → None

Oublier le return est LE bug silencieux — l'annotation -> le rend visible.



tp_penalites.py

```
def taux_penalite(jours: int,  
                  plafond: float = 0.50) -> float:  
    """Taux applicable, plafonné."""  
    if jours <= 0:  
        return 0.0  
    taux = 0.10 + 0.02 * mois(jours)  
    return min(taux, plafond)  
  
taux_penalite(45) # 0.12  
taux_penalite(45, plafond=0.3) # nommé
```

*args, **kwargs et le déballage



*args ramasse les positionnels

`total(*montants)` : la fonction accepte 2 ou 200 valeurs, reçues en tuple.



**kwargs ramasse les nommés

Utile pour transmettre des options telles quelles à une autre fonction.



* déballe à l'appel

`coords = ("BIS", "Bissau") ; afficher(*coords)` — la séquence devient des arguments.



À doses mesurées

Une signature explicite reste préférable quand les paramètres sont connus d'avance.



*args / **kwargs

```
def total(*montants: int) -> int:  
    return sum(montants)
```

```
total(8_100_000, 2_500_000)  
10600000
```

```
mensuels = [100, 200, 300]  
total(*mensuels)      # déballage  
600
```

```
def exporter(df, **options):  
    df.to_csv("out.csv", **options)
```

Portée : qui voit quoi — et LE piège classique



Local d'abord (règle LEGB)

Une variable créée dans la fonction meurt avec elle — c'est une protection.



Lire une globale : oui, la modifier : non

Passez-la en paramètre et renvoyez le résultat — testable et lisible.



JAMAIS de défaut mutable

def f(x=[]): le défaut est créé UNE fois et partagé entre tous les appels.



L'idiome de remplacement

x=None puis if x is None: x = [] — ou default_factory en dataclass (J4).



le défaut mutable

```
# LE piège
def ajouter(d, liste=[]):      # x
    liste.append(d)
    return liste

ajouter("a")    # ["a"]
ajouter("b")    # ["a", "b"]  !!

# l'idiome correct
def ajouter(d, liste=None):    # ✓
    if liste is None:
        liste = []
```

assert : verrouiller les bords du barème



Un test en une ligne

`assert f(30) == 0` — si faux, arrêt net avec la ligne fautive à l'écran.



Les bords d'abord

30 vs 31 jours, zéro, le plafond : les bugs vivent aux frontières des règles.



Comparer des floats

`abs(f(65) - 0.14) < 1e-9` — jamais `==` entre deux floats (cf. J1).



Le TP fournit les assert

Votre code doit les faire passer : c'est le contrat — l'antichambre de pytest (J5).



les bords

```
# Règle : 10 % + 2 %/mois entamé  
# au-delà du 1er mois, plafond 50 %  
  
assert mois_entames(30) == 0  
assert mois_entames(31) == 1  
assert mois_entames(61) == 2  
  
assert taux_penalite(0) == 0.0  
assert abs(taux_penalite(65) - 0.14) < 1e-9  
assert taux_penalite(2000) == 0.50  
  
$ python tp_penalites.py # silence = OK
```

Les fonctions sont des valeurs : lambda et key=



Une fonction se passe en argument

`sorted(annuaire, key=extraire_nom)` — le tri délègue le « comment comparer ».



lambda : la mini-fonction anonyme

`key=lambda c: c["nom"]` — une expression, pas plus ; sinon un `def` nommé.



Tri multi-clés avec un tuple

`key=lambda c: (c["region"], c["nom"])` : région PUIS nom, en une passe.



max et min aussi

`max(declars, key=lambda d: d["ca"])` : la plus grosse déclaration.



`sorted, key, lambda`

```
tries = sorted(annuaire,
               key=lambda c: c["nom"])

# région PUIS nom
tries = sorted(annuaire,
               key=lambda c: (c["region"],
                             c["nom"]))

# la plus grosse déclaration
top = max(declars,
          key=lambda d: d["ca"])
```

Listes : découpage, copie, mutation



Le slicing lit par tranches

lst[:5] les 5 premiers • lst[-3:] les 3 derniers • lst[::2] un sur deux.



b = a ne copie PAS

Deux noms, une liste : muter b mute a. Copier : a.copy() — ou a[:] en idiome court.



sort() vs sorted()

lst.sort() modifie sur place et renvoie None ; sorted(lst) renvoie une nouvelle liste.



append vs extend

append(xs) ajouterait la liste ENTIÈRE comme un élément ; extend(xs) ajoute chaque élément.



listes

```
cas = [45, 620, 150, 98, 510]
cas[:3]      # [45, 620, 150]
cas[-2:]     # [98, 510]
cas[::2]     # [45, 150, 510]

b = cas      # MÊME liste
b.append(999) # cas aussi !
c = cas.copy() # indépendante

cas.sort()   # sur place → None
tries = sorted(cas) # nouvelle liste
```


Dictionnaires : index, parcours, comptage



d[cle] plante, d.get() jamais

get(cle, default) : LE réflexe face aux données incomplètes.



Parcourir avec items()

for region, n in compte.items(): — clé et valeur ensemble, dépaquetées.



Le motif du comptage

compte[k] = compte.get(k, 0) + 1 — à connaître par cœur.



L'index par clé

par_nif = {c["nif"]: c for c in annuaire} : accès direct $O(1)$, fini les boucles de recherche.



tp_annuaire.py

```
# comptage par région
compte = {}
for c in annuaire:
    r = c["region"]
    compte[r] = compte.get(r, 0) + 1

# affichage trié par effectif
for region, n in sorted(
    compte.items(),
    key=lambda kv: -kv[1]):
    print(f"region:<12}{n:>6}")
```

collections : Counter et defaultdict



Counter compte tout seul

Counter(c["region"] for c in annuaire) — le motif get(k,0)+1, industrialisé.



most_common(n)

Le top 3 des régions en un appel — trié, prêt à afficher.



defaultdict(list) regroupe

par_region[r].append(c) sans tester l'existence de la clé.



Savoir revenir au dict nu

Ces outils optimisent un motif que vous savez déjà écrire à la main.



collections

```
from collections import Counter, \
    defaultdict

compte = Counter(c["region"]
                 for c in annuaire)
compte.most_common(3)
[('Bissau', 2), ('Bafatá', 2), ...]

par_region = defaultdict(list)
for c in annuaire:
    par_region[c["region"]].append(c)
```

Ensembles : unicité et algèbre



Unicité garantie, "in" instantané

Tester l'appartenance dans un set est $O(1)$ — dans une liste, on parcourt tout.



Le dédoublonnage type

Un set des NIF vus + une liste des fiches gardées (l'ordre est préservé).



$a - b$, $a \& b$, $a | b$

Différence : déclarants inconnus. Intersection : communs. Union : tous.



Cas réel du dépôt

paiements.csv contient des NIF orphelins — une soustraction les révèle.



algèbre des ensembles

```
vus, uniques = set(), []
for c in annuaire:
    if c["nif"] not in vus:
        vus.add(c["nif"])
        uniques.append(c)

declarants = {"...001", "...009"}
connus = {c["nif"] for c in uniques}

declarants - connus # orphelins
declarants & connus # communs
```

Quelle structure pour quel besoin ?

Besoin	Structure	Exemple du fil rouge
Une suite ordonnée, modifiable	liste []	les déclarations d'un contribuable
Retrouver par clé en $O(1)$	dict { : }	l'index nif → fiche
Unicité, appartenance, comparaison	set { }	NIF vus, NIF orphelins
Un enregistrement figé	tuple ()	(code_region, nom_region)
Compter par catégorie	Counter	contribuables par région
Regrouper par catégorie	defaultdict(list)	fiches par région
Fichier CSV en mémoire	liste de dicts	l'annuaire entier

Règle d'or : une boucle qui CHERCHE un élément cache presque toujours un dict ou un set qui manque.

Pénalités et mini-annuaire

- 1 **tp_penalites.py** — faire passer tous les assert — les bords d'abord
- 2 **tp_annuaire.py** — comptage, dédoublonnage, orphelins, tri multi-clés
- 3 **Refactorer son J1** — extraire les répétitions en fonctions annotées
- 4 **Pour les rapides** — version Counter/defaultdict + index par NIF en compréhension



Fichiers du jour

```
03-fonctions/  
    tp_penalites.py  
04-structures-de-donnees/  
    tp_annuaire.py
```



Un assert qui échoue vous DIT où chercher :
reproduisez le cas au REPL, corrigez, relancez.

À retenir

- ✓ **Signatures** — nommés + défauts + annotations : l'appel se lit sans ouvrir la fonction.
- ✓ **Jamais de défaut mutable** — `x=None` puis `if x is None` — le piège est désamorcé une fois pour toutes.
- ✓ **`key=lambda`** — tris multi-clés, max, min : la fonction passée en valeur.
- ✓ **dict/set en $O(1)$** — index par NIF, orphelins en une soustraction — fini les boucles de recherche.
- ✓ **Counter / defaultdict** — les motifs de comptage et de regroupement, industrialisés.



Demain — J3 : fichiers, formats et robustesse

pathlib, CSV et JSON réels, hiérarchie des exceptions, exceptions métier personnalisées — et votre premier paquet Python.